CAMBIOS DE GESTIÓN Y DOCENCIA

HitSchool
Así funciona ahora

Notas, tareas, chat, matrículas y pagos

Guía detallada de los flujos de pantalla, contratos de API, reglas de cálculo, persistencia y controles añadidos.

Del gesto de la profesora al dato guardado

Cada bloque explica qué hace la persona usuaria, qué petición envía la interfaz, qué decide el backend y qué información conserva el modelo de datos. Incluye ejemplos, situaciones límite y referencias al código.

Implementado no significa desplegado

Esta guía describe la copia de trabajo revisada el **13 de noviembre de 2026**. La persistencia y la migración se probaron en PostgreSQL temporal y se añadieron datos rotulados de prueba a la base Docker local. **Las cuentas de acceso y la base del servidor no se modificaron.**

Edición y alcance
13/11/2026 · Incluye fixture local; PostgreSQL del servidor remoto pendiente.

Audiencia
Administración, profesorado y mantenimiento técnico de la plataforma.

MAPA DEL DOCUMENTO

01 · LECTURA Y ALCANCE

Qué ha cambiado

El objetivo no es sustituir los cálculos automáticos, sino añadir flexibilidad docente y hacer coherentes la pantalla, la API y el dato persistido. En pagos se han corregido fallos concretos sin rediseñar el sistema de facturación.

Área	Cambio principal	Páginas
Etiquetas	«Midterm» en expedientes, vistas de notas y boletines. Sin renombrar campos internos.	4, 6, 8
Cálculos y medias	Valores manuales separados de automáticos, incluido el cero, y reinicio individual.	4–7
Guardado de notas	Devuelve competencias antes omitidas; una misma resolución al guardar y consultar.	7–10
Tareas y exámenes	Edición numérica, conservación al guardar comentarios y modo de nota global de tarea.	8–10
Chat	Negrita solo para pendientes y lectura limitada a mensajes realmente cargados.	11–12
Alta y pagos	Alta transaccional, protección de cargos especiales y controles al registrar cobros.	13–17
Entrega técnica	Migración aditiva, pruebas repetibles, datos locales de verificación, límites y mapa de archivos.	18–22

Lectura funcional

Empieza por las páginas 6, 11, 13, 15 y 17: muestran acciones, efectos y ejemplos de uso.

Lectura técnica

Continúa con contratos API, modelos y transacciones en las páginas 7–10, 12, 14 y 18.

Decisiones y estado local

- Curso de notas por defecto: `2025-2026`.
- Alta: genera deuda pendiente; el cobro se marca después.
- La tarifa trimestral se introduce como importe mensual.
- No se ha cambiado la incorporación a mitad de trimestre.
- Docker local está activo; la base contiene registros demo identificados. El servidor remoto no se ha tocado.

Los estados «probado» de esta guía corresponden a pruebas simuladas o navegador con API interceptada, salvo que se indique lo contrario. No equivalen a una certificación del entorno desplegado.

02 · DEL NAVEGADOR A PRISMA

Un flujo, varias capas

React
Formulario y estado de edición

→

HTTP + JWT
Identidad, rol y petición JSON

→

Express
Validación y reglas de negocio

→

Prisma
Lecturas, escrituras y transacciones

La interfaz no sustituye al servidor

El editor muestra cálculos para facilitar la edición, pero el backend valida y resuelve las notas efectivas. Al guardar correctamente se vuelve a consultar el expediente para recuperar su representación completa. Los permisos no dependen de que un botón esté oculto.

Rol	Notas	Chat
`ADMIN`	Edición de las clases y entregas permitidas por las rutas administrativas.	Interlocutor de alumnos y tutores; usa la pantalla compartida del personal.
`TEACHER`	Presenciales accesibles; online propias o asignadas. Las entregas verifican su acceso.	Interlocutor de alumnos y tutores.
`STUDENT`	Consulta de sus notas; no puede usar las rutas de escritura docente.	Recibe y envía mensajes al personal.
`PARENT`	Consulta con comprobación del vínculo familiar.	Conversaciones contextualizadas por hijo.

Entidades que no conviene confundir

`Enrollment`	Vínculo del alumno con una clase; interviene en las notas de ese curso.
`AcademyEnrollment`	Matrícula económica: fecha de alta, tarifa, periodicidad, modalidad y libro.
`Submission` / `TaskDelivery`	Nota del paso o entrega individual / nota global y feedback de una tarea.
`TermGrade`	Evaluación por alumno, clase, trimestre y curso académico.
`PaymentStatus`	Obligación de pago y estado de cobro; no es una operación bancaria.

Base técnica: React 19 + TypeScript; Express; Prisma con PostgreSQL. Las factorías de rutas permiten probar los mismos handlers con una dependencia de datos simulada.

03 · LAS REGLAS DE PARTIDA

Cómo salen las medias

Midterm y Final en presencial

La media del examen se calcula con las competencias informadas: Grammar, Reading, Writing, Listening y Speaking. No se convierte una competencia vacía en cero. Si no hay competencias, se conserva la compatibilidad con la nota de examen existente cuando corresponda.

Trimestral = 0,35 × Midterm + 0,35 × Final + 0,30 × Tareas

La fórmula presencial automática necesita ambas notas de examen. Si falta alguna, la global automática queda sin calificar. Si existen los dos exámenes pero no hay media de tareas, la fórmula actual utiliza cero para ese componente; **no se han redistribuido los porcentajes**.

Ejemplo presencial

Midterm = 8; Final = 9; tareas = 7.

2,80 + 3,15 + 2,10 = 8,05.

Un valor manual superior puede sustituir este resultado; no altera los porcentajes guardados en el código.

Pasos → tarea → trimestre

Para el cálculo trimestral, una tarea automática aporta su media cuando están calificados todos sus pasos evaluables. Un paso evaluable exige entrega o contiene un formulario.

Online: competencias y evaluación continua

1. Se calcula la media de tareas por competencia reconocida.
2. La global automática es la media de las competencias que tienen nota, tomando sus valores efectivos.
3. Si no hay competencias disponibles, se utiliza la media de tareas como alternativa.
4. Si se fija manualmente `tasksAverage`, ese valor pasa a ser la base de la global automática online. Una global manual sigue teniendo prioridad.

Por qué 10, 10 y 0 pueden dar 5

Dos tareas de Grammar con 10 y una de Reading con 0 dan Grammar = 10 y Reading = 0. La global por competencias es **(10 + 0) / 2 = 5**; la media de las tres tareas sería 6,67. Ahora guardar y consultar utilizan la misma resolución que la vista automática existente.

Categorías: `GRAMMAR_VOCABULARY` alimenta Grammar; Reading, Writing, Listening y Speaking tienen sus categorías propias. `MOCK_EXAM` y categorías no mapeadas pueden contribuir a la media de tareas, pero no crean una sexta competencia. En las ramas de notas, `HIBRIDO` sigue el cálculo no online.

Redondeo habitual del backend: dos decimales. Algunas vistas y el boletín presentan un decimal. La presentación no cambia el valor que usa la API.

04 · AUTOMÁTICA Y MANUAL NO SON EL MISMO DATO

Qué valor prevalece

Valor efectivo = ajuste manual, si existe; si no, valor automático

El ajuste se guarda en `TermGrade.gradeOverrides`. No se deduce que una nota sea manual simplemente porque difiera de la media. La API devuelve tanto la resolución efectiva como la información necesaria para distinguir los modos.

Campo	Significado
`tasksAverage`, `overallGrade`…	Valores efectivos que se muestran y usan como resultado.
`automaticGrades`	Cálculos disponibles para cada campo. Una media superior puede incorporar ajustes manuales de sus componentes.
`gradeOverrides`	Solo los ajustes explícitos vigentes: número entre 0 y 10.

Un cero manual no es un dato vacío

La resolución utiliza una comprobación nula (`??`), no una comprobación de verdad/falsedad. Por ello, un ajuste de **0** no desaparece ni se sustituye por una media automática.

Petición	Efecto sobre el ajuste
`7.5`	Guarda 7,5 como valor manual.
`0`	Guarda cero como valor manual válido.
`null`	Elimina el ajuste de ese campo: vuelve a automática.
Clave omitida	Conserva el ajuste existente de ese campo.

Los ajustes son independientes

Si la profesora cambia la media de Midterm, la global automática usa ese Midterm efectivo. Si además fija una global manual, esa global no se mueve aunque cambien competencias, exámenes o tareas. Reiniciar la media de tareas no borra el ajuste de la global.

«Automática» es local a cada campo

Volver a automática en la global recalcula con los componentes efectivos actuales. No elimina los ajustes manuales que sigan activos en Midterm, Final, tareas o competencias.

05 · UN CASO DE CLASE PRESENCIAL

Editar sin perder el cálculo

1. **Abrir el expediente de la clase.** La pantalla carga el alumno, la clase y el trimestre seleccionado. La consulta incluye `courseId` para evitar tratar la evaluación de varias clases como una sola.
2. **Pulsar «Editar Evaluación».** El formulario recupera las competencias, los valores efectivos y los ajustes manuales. La edición se bloquea si ha fallado la carga inicial.
3. **Introducir la nota necesaria.** Competencias y notas directas siguen siendo editables; las medias muestran el modo «Manual» o «Automática».
4. **Guardar.** Se envían las competencias, observaciones y cambios explícitos de `gradeOverrides`. La API valida y persiste la evaluación.
5. **Recargar la evaluación.** Tras éxito, el editor se cierra y se consulta de nuevo el expediente completo. Una respuesta de error no debe aparentar un guardado correcto.

Alumno que hace las tareas en el aula

Puede no tener entregas online con las que calcular una media. La profesora introduce, por ejemplo, **7 en «Media de tareas»**. En presencial, si Midterm es 8 y Final es 9, la global automática resulta 8,05. El trabajo presencial no exige inventar entregas para fijar ese componente.

Más tarde aparecen entregas online

Su media automática puede ser 6. El ajuste manual de tareas continúa en 7 hasta pulsar **«Volver a automática» en ese campo y guardar**. Entonces se elimina la clave manual y la global usa el 6, salvo que la propia global tenga otro ajuste.

Qué cambia visualmente

- «Middle Term» pasa a **«Midterm»**; los campos `middle*` no se renombran.
- Se editan medias de Midterm/Final, tareas, global y competencias online calculadas.
- Los errores de carga y guardado se muestran de forma visible.
- Se retira la media local de respaldo que mezclaba entregas de otros trimestres.
- Los controles se adaptan a escritorio y móvil; se ha probado su uso en ambos.

NOTAS · CONTRATO HTTP

06 · LEER Y ESCRIBIR LA MISMA EVALUACIÓN

API de calificaciones

Ruta	Responsabilidad
`GET /api/term-grades/course/:courseId`	Matriz de la clase. Acepta trimestre y curso académico.
`GET /api/term-grades/student/:studentId`	Expediente del alumno; la pantalla envía `?courseId=…`.
`PUT /api/term-grades/course/:courseId`	Guarda competencias, observaciones y ajustes del alumno y trimestre.

Se requiere `Authorization: Bearer <token>`. Los ejemplos son parciales: no sustituyen todos los campos que devuelve una consulta real.

```
PUT /api/term-grades/course/clase-id
{
  "studentId": "alumno-id",
  "term": 1,
  "academicYear": "2025-2026",
  "middleGrammar": 8,
  "observations": "Trabajo realizado en clase",
  "gradeOverrides": {
    "tasksAverage": 7,
    "overallGrade": null
  }
}
```

El ejemplo fija tareas en 7 y elimina solo el ajuste de la global. Las otras claves omitidas se conservan. Se comprueba que el alumno está matriculado en la clase antes de crear o actualizar su `TermGrade`.

```
Respuesta conceptual, con el resto de datos ya informados:
{
  "tasksAverage": 7,
  "overallGrade": 8.05,
  "gradeOverrides": { "tasksAverage": 7 },
  "automaticGrades": {
    "tasksAverage": 6,
    "overallGrade": 8.05
  }
}
```

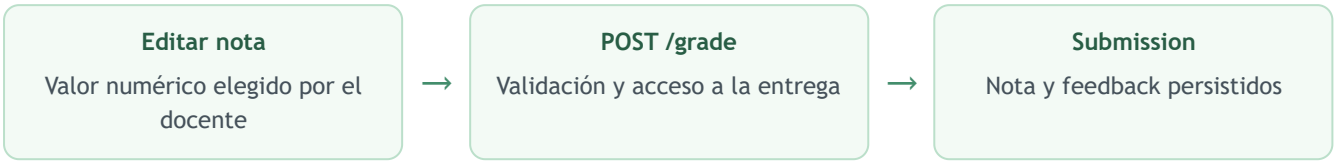
Compatibilidad y validación

Con un objeto `gradeOverrides` explícito, los cambios de calculados se toman de ese objeto. Sin él, la ruta admite campos calculados planos por compatibilidad. Se valida trimestre 1–3, notas finitas 0–10 y claves de ajuste permitidas. Las lecturas y la respuesta del guardado usan `resolveTermEvaluation`.

Campos ajustables: middleExamGrade, finalExamGrade, tasksAverage, overallGrade, grammar, reading, writing, listening y speaking.

07 · LA NOTA NO ES LA RESPUESTA DEL ALUMNO

Calificar y comentar




```
POST /api/assignments/submissions/entrega-id/grade
{ "grade": 3.5, "feedback": "Revisado presencialmente" }

Solo comentarios, sin sustituir la nota:
{ "feedback": "Añadida una aclaración" }
```

Qué se ha corregido

- Los exámenes disponen de acceso a **«Editar nota»**, además de revisar sus respuestas.
- El formulario numérico ya no impone de nuevo la puntuación autocorregida al guardar una nota elegida por el docente.
- Si `grade` no se envía, el backend conserva la nota actual al guardar feedback.
- La conversión usa `Number`; una cadena como `"8invalid"` no se acepta como un 8 parcial.
- Los ceros se guardan como notas válidas; un valor explícitamente vacío o nulo no significa lo mismo que omitir la clave.

Permisos efectivos, no solo botones

`requireTeacher` admite `ADMIN` y `TEACHER`. Para un profesor se verifica además la asignación: autor de la tarea, clase presencial, titular de la clase o docente asignado. Una entrega sin acceso devuelve 403 antes de actualizar la nota. El administrador mantiene su acceso administrativo.

Revisión de preguntas y edición numérica son rutas distintas

La revisión de examen puede enviar `questionScores` y actualizar la puntuación guardada en el contenido JSON. La edición numérica cambia `Submission.grade`; no vuelve a entregar el examen ni cambia por sí sola las respuestas del alumno.

Si pertenece a una tarea estructurada, el guardado invoca la sincronización de su entrega global. Esta respeta el modo manual/automático de la tarea.

El botón «Volver a automática» de esta intervención afecta a medias calculadas y a la nota global de tarea; no es un botón para repetir el examen del alumno.

08 · UN BLOQUE DOCENTE COMPLETO

Manual o automática

El expediente permite abrir **«Evaluar tarea»** cuando dispone de los datos de esa tarea del trimestre. El modal reúne los pasos, sus notas y comentarios, la nota global y el feedback del bloque. También permite editar la nota de un paso de examen.

`gradeMode: AUTO`

La nota efectiva de la tarea se calcula desde sus pasos evaluables. Para la media trimestral deben estar todos calificados. Los cambios posteriores de esos pasos se reflejan en la nota automática.

`gradeMode: MANUAL`

Se usa la nota global que introduce el docente. Puede ser cero. No se sustituye por la media de pasos al sincronizar la tarea.

```
POST /api/structured-tasks/tarea-id/grade-delivery
{
  "studentId": "alumno-id",
  "grade": 9,
  "gradeMode": "MANUAL",
  "feedback": "Evaluación global presencial",
  "stepEvaluations": [
    { "stepId": "paso-id", "grade": 5, "feedback": null }
  ]
}
```

1. Se valida la tarea accesible para el docente y el rango de todas las notas enviadas.
2. En una transacción se crean o actualizan las entregas de pasos y su feedback.
3. Se resuelve el modo global. En AUTO se toma la media; en MANUAL se exige una nota.
4. Se guarda `TaskDelivery.grade`, `gradeMode`, estado y fecha de calificación.
5. Al consultar la evaluación trimestral, una tarea manual aporta su nota global, no una media alternativa que la ignore.

Volver a automática, de verdad

Con pasos 6 y 8, la automática es 7. Una global manual de 9 prevalece. Si se pulsa «Volver a automática» y se guarda, se persiste AUTO; si después el segundo paso pasa a 10, la nota automática será 8.

Compatibilidad: una tarea antigua con modo nulo y estado GRADED conserva su nota como manual hasta un reinicio explícito. No se intenta adivinar ni reescribir todo el historial durante la migración.

09 · GUARDAR NO BASTA: HAY QUE PODER RELEER

Evitar pérdidas silenciosas

El fallo de las competencias

El guardado ya persistía las diez competencias de Midterm y Final, pero las respuestas de lectura no las incluían. Al reabrir aparecían vacías; un nuevo guardado podía convertir esos vacíos en nulos. Ahora las consultas de clase y alumno devuelven esos diez campos.

Familia de campos	Datos conservados en la lectura
`middle*`	Grammar, Reading, Writing, Listening y Speaking de Midterm.
`final*`	Las cinco competencias del examen Final.
Medias y global	Valores efectivos, automáticos y ajustes explícitos.
Observaciones y tareas	Texto docente y desglose de tareas que fundamenta la evaluación.

El recorrido que se ha comprobado

Guardar — PUT con competencias y ajustes → **Consultar** — GET de clase y alumno → **Reabrir** — Valores completos, incluido cero → **Comentar** — Sin borrar notas omitidas

Las pruebas HTTP recorren el mismo router y autenticación que usa la aplicación, con una dependencia de almacenamiento simulada. El navegador comprueba que los controles recuperan el valor tras reabrir. Esto acredita la coherencia del flujo probado, **no todavía una escritura en PostgreSQL desplegado**.

Claves de aislamiento

- `TermGrade` se identifica por alumno + clase + trimestre + curso académico.
- Las rutas de expediente usadas por la interfaz envían la clase concreta.
- Las notas formales mantienen el año por defecto `2025-2026`, por decisión expresa.
- La media local que mezclaba entregas de diferentes trimestres deja de actuar como sustituto silencioso cuando la API no tiene media.

Alcance anual: el año filtra los registros formales `TermGrade`. Las tareas se seleccionan por clase y trimestre; esta entrega no añade un filtro anual nuevo a las tareas. Reutilizar la misma clase entre cursos exige comprobar su histórico.

Precaución para nuevos consumidores de la API

Un listado histórico o un cuadro global no debe interpretar una evaluación multiclase como si fuera la de una clase concreta. Para editar y verificar un expediente utiliza siempre el `courseId` correcto y un año explícito cuando corresponda.

CHAT · COMPORTAMIENTO VISIBLE

10 · CUÁNDO UN MENSAJE PASA A LEÍDO

Negrita con significado

La lista de conversaciones es compartida por los cuatro roles. Antes todos los nombres aparecían destacados; ahora el estilo refleja el contador de mensajes recibidos pendientes de ese hilo.

Situación	Resultado
`unreadCount > 0`	Nombre en negrita, peso 700, con contador de pendientes.
`unreadCount = 0`	Peso normal, 400. La selección de la conversación mantiene su propio fondo.
Abrir solo el listado	No marca mensajes como leídos.
Falla la carga del hilo	No se limpia el contador como si se hubieran mostrado los mensajes.
Se carga el hilo visible	Se solicita marcar los mensajes entrantes cargados que continúan sin leer.

1. Se elige un contacto y se calcula su contexto de alumno, cuando existe.
2. Se consulta el historial. Las respuestas tardías de otra conversación se descartan mediante la clave activa y el identificador de petición.
3. Después de renderizar, si el documento está visible, se seleccionan mensajes entrantes sin `readAt`.
4. Se envían sus identificadores al servidor. Solo después de una respuesta correcta se actualiza el estado local y se refrescan contactos.

Mensajes nuevos con el hilo abierto

El polling vuelve a cargar cada 3,5 segundos. El efecto de mensajes vuelve a aplicar el marcado cuando el documento está visible. Los eventos de foco y cambio de visibilidad permiten retomar el proceso. Los identificadores en curso evitan duplicaciones simultáneas de la misma solicitud.

Qué significa «visible» aquí

Se comprueba la visibilidad de la pestaña/documento y que el hilo se haya cargado. No se certifica que cada burbuja individual haya entrado en el área visible por scroll. La negrita se aplica al listado, no a las burbujas.

11 · NO LEER LO QUE TODAVÍA NO SE HA MOSTRADO

Lectura por identificadores

```
GET /api/chat?partnerId=docente-id&studentId=hijo-id

POST /api/chat/read
{
  "partnerId": "docente-id",
  "studentId": "hijo-id",
  "messageIds": ["mensaje-1", "mensaje-2"]
}

Respuesta: { "unreadCount": 1 }
```

La respuesta anterior no tiene por qué ser cero: puede haber llegado un mensaje posterior a la carga. El backend actualiza solo los identificadores enviados que coinciden con el emisor, el destinatario autenticado, el contexto del alumno y `readAt: null`.

Ejemplo de una carrera que ya no limpia mensajes nuevos

1. El GET devuelve mensajes 1 y 2.
2. Llega el mensaje 3 antes del POST de lectura.
3. La interfaz envía únicamente 1 y 2.
4. El 3 queda pendiente: `unreadCount = 1` hasta cargarlo y procesarlo.

Conversaciones de familias

La clave de agrupación no es solo la persona: incluye el alumno contextual. Un tutor puede tener dos hijos y mantener dos hilos con el mismo docente. El estado local ya no pone a cero todos los contactos con ese interlocutor. El envío y consulta tutor-administrador conservan ahora el mismo contexto que tutor-profesor.

Ruta o campo	Función
`GET /api/chat/contacts`	Contactos, contador por hilo y última actividad.
`GET /api/chat/unread-count`	Total pendiente del usuario, empleado por los avisos del layout.
`ChatMessage.readAt`	Persistencia de lectura por mensaje y destinatario.
`messageIds`	Lista obligatoria; como máximo 1.000 identificadores por petición de lectura.

No se ha hecho una migración masiva de mensajes antiguos sin contexto de alumno ni un rediseño general de los permisos de conversación. Los avisos globales conservan su actualización por polling.

12 · CREAR DEUDA NO ES COBRAR DINERO

Del alumno a sus cuotas

Ficha
Usuario alumno inicialmente inactivo

→

Dar de alta
Matrícula económica y activación

→

Cuotas
Importes pendientes

→

Registrar cobro
Acción posterior desde pagos

Primera pantalla: ficha del alumno

`POST /api/students` crea la identidad, perfil y, cuando procede, el vínculo con un tutor. Esta operación no equivale por sí sola a crear una matrícula económica ni a registrar un pago recibido.

Segunda acción: formalizar el alta

El modal envía fecha, modalidad, periodicidad e importe. La ruta `POST /api/enrollments/enroll` crea la matrícula y genera los registros que correspondan. Nacen con `status: PENDING` e `isPaid: false`.

Cómo interpretar el importe

Tarifa de ejemplo	Interpretación real
Mensual: 65 €	65 € por cada mes ordinario generado, salvo reglas específicas como septiembre.
Trimestral: 60 €	60 € por cada mes del trimestre generado. Octubre, noviembre y diciembre completos suman 180 €.
Precio del libro	Concepto separado, si está activado y tiene un precio positivo.

60 y 65 no son precios obligatorios. Son ejemplos de tarifas editables. El campo interno `monthlyFee` continúa representando el importe mensual también con periodicidad `QUARTERLY`; no se divide entre tres ni añade por sí solo un descuento.

Sin cobro bancario automático

Estas rutas registran obligaciones y la confirmación administrativa del cobro. No se ha añadido una pasarela, domiciliación, cargo en tarjeta, pago parcial ni un cobro inicial dentro del alta.

MATRÍCULAS · ALTA TRANSACCIONAL

13 · NO DEJAR UNA ACTIVACIÓN A MEDIAS

Todo el alta o nada

Antes, crear matrícula, activar al alumno y generar cuotas eran operaciones separadas. Un error podía dejar un alumno activo sin calendario completo, y el reintento normal se rechazaba por estar ya activo.

```
POST /api/enrollments/enroll
{
  "studentId": "alumno-id",
  "monthlyFee": 60,
  "billingPeriod": "QUARTERLY",
  "modality": "PRESENCIAL",
  "startDate": "2026-10-01",
  "hasBook": true,
  "bookPrice": 35
}
```

1. **Validación previa.** Identificador, importe finito positivo, fecha válida, periodicidad permitida, modalidad y precio del libro.
2. **Preparar contraseña.** Se genera la temporal y su hash antes de entrar en la transacción.
3. **Consultar y activar.** Dentro de `prisma.$transaction` se verifica el alumno y se hace una actualización condicional para activar solo si aún no está activo.
4. **Crear matrícula y actualizar cuenta.** Se guardan tarifa, modalidad, periodicidad y credenciales cifradas.
5. **Generar cuotas con la misma transacción.** `ensureStudentPaymentScheduleById` acepta el cliente transaccional.
6. **Confirmar y notificar.** El SMTP se intenta después del commit: bienvenida en primera matrícula, reactivación en las siguientes.

Si falla una cuota

La transacción revierte también activación y matrícula. La prueba HTTP inyecta ese fallo y verifica que no se envía la notificación.

Si falla el correo

El alta confirmada se conserva. El fallo se registra; no se deshacen matrícula ni pagos. La entrega del email requiere comprobación SMTP separada.

Una repetición del alta de un alumno ya activo se rechaza; la activación condicional también evita que dos procesos activen la misma cuenta como si ambos fueran el primero. Las pruebas reales de concurrencia en PostgreSQL siguen pendientes.

14 · LO QUE GENERA EL SERVICIO

Meses, septiembre y fechas

Regla	Funcionamiento actual
Mensual	Recorre desde el mes inicial hasta el fin de matrícula o el mes actual. Un inicio futuro contempla al menos su mes inicial.
Trimestres	Octubre–diciembre, enero–marzo y abril–junio. Se crean cuotas mensuales separadas, no un cargo único por trimestre.
Septiembre trimestral	Se trata por separado y el generador también prepara octubre–diciembre.
Julio y agosto	El generador ya no crea nuevas cuotas ordinarias de esos meses. No borra automáticamente registros que ya existan ni prohíbe un concepto extraordinario explícito.
Curso económico	Se calcula a partir de septiembre. No se ha cambiado al valor fijo de notas.

La regla del 15 de septiembre

La cuota se reduce al 50 % si el mes generado es septiembre del año de inicio, el alta empezó en septiembre, el día de inicio es 15 o posterior y la modalidad es exactamente `PRESENCIAL`.

Ejemplos: 65 € → 32,50 €; 60 € → 30 €. Antes del día 15, en otro mes, online o híbrido no se aplica esta reducción. No es un prorrateo diario general.

Vencimiento e impago visual

- El vencimiento usa el día del alta a las 12:00 locales.
- Si ese día no existe, se ajusta al último del mes: por ejemplo, un día 31 en febrero.
- Con fecha válida, se considera vencido al superar el vencimiento más diez días.
- Pagados y exentos no se clasifican como impago. `OVERDUE` es un estado visual calculado; no es un nuevo valor persistido del enum de pagos.

Incorporación a mitad de trimestre: sin cambios

Se ha respetado la exclusión pedida. El helper conserva los meses restantes del trimestre inicial, pero el recorrido posterior puede omitir enero si empezó en noviembre y avanza en saltos de tres meses. Completar bloques también puede incluir meses posteriores a una fecha de fin intermedia. No se confirma ese calendario completo; requiere revisión aparte.

15 · CAMBIAR TARIFA SIN CAMBIAR EL LIBRO

Qué se recalcula y qué no

La edición de la tarifa desde alumnos actualizaba de forma amplia los importes de pagos no cobrados. Eso podía pisar libros, conceptos personalizados o cuotas exentas. Ahora actualiza la tarifa de matrícula y delega la sincronización en el generador, que conoce esas excepciones.

Registro existente	Tratamiento durante la sincronización
Cuota ordinaria no pagada	El importe puede cambiar si corresponde al mes actual o a uno futuro. Su vencimiento puede sincronizarse.
Cuota de un mes pasado	No se reescribe su importe solo por cambiar ahora la tarifa.
`isPaid = true`	No se recalcula el importe ni el vencimiento por esta sincronización ordinaria.
`isCustom = true`	Se conserva el importe personalizado.
`isExempt = true`	Se conserva la exención y no se transforma en deuda ordinaria nueva.
Eliminado expresamente	El registro de eliminación evita que el generador lo recree automáticamente.

Ejemplo de modificación de tarifa

Hay un mes pagado por 17 €, un mes personalizado de 21 €, uno exento de 0 € y una cuota ordinaria futura de 65 €. Al cambiar la tarifa a 70 €, las pruebas conservan **17, 21 y 0**; solo la última pasa a **70**.

Libro y conceptos extraordinarios

El alta puede crear «Material / Libro» como concepto separado y personalizado. La pantalla conserva herramientas para añadir conceptos, editar importes puntuales, eximir y eliminar. Esta intervención no ha rediseñado su facturación ni convertido un libro en cuota mensual.

Consultar pagos puede escribir datos

Las rutas de consulta financiera ejecutan sincronización para generar o completar cuotas. Abrir la pantalla de pagos no equivale necesariamente a una operación de solo lectura sobre la base de datos. Por eso las pruebas reales se harán en un destino separado.

La idempotencia secuencial se ha probado en memoria: repetir la sincronización no duplica esas cuotas de prueba. La concurrencia real y las matrículas históricas requieren PostgreSQL.

16 · ESTADOS Y OPERACIONES ADMINISTRATIVAS

Registrar el dinero recibido

```
PUT /api/payments/alumno-id
{ "paymentId": "cuota-id", "isPaid": true }
```

Acción	Actualización
Marcar pagado	`isPaid=true`, `status=PAID`, fecha de servidor `paidAt` y usuario en `markedById`.
Desmarcar	`isPaid=false`, `status=PENDING` y `paidAt=null`. La deuda puede volver a verse vencida si supera su fecha.
Cuota de otro alumno	404, sin modificarla: se comprueba la pertenencia al alumno de la URL.
Cuota exenta	400: no se registra como cobrada mientras esté exenta.

«Marcar todo pagado» no significa solo meses vencidos

`PUT /api/payments/:studentId/pay-all` marca los registros pendientes no exentos del alumno. Con curso seleccionado, filtra por ese curso. Incluye cuotas futuras que ya estén generadas; no se ha añadido una selección por rango ni un pago parcial.

Funciones conservadas del documento de referencia

Herramienta	Implementación existente / alcance de esta entrega
Editar importe puntual	`PUT /api/payments/item/:id`; deja el concepto como personalizado.
Eliminar completamente	`DELETE /api/payments/item/:id`; conserva memoria de la eliminación para no regenerar.
Eximir	Marca el registro como exento; no cuenta como pendiente ordinario.
Añadir concepto	`POST /api/payments/:studentId/extra`; herramienta administrativa existente.
Facturas, bajas y selector de curso	Se mantienen las pantallas y generación de recibos existentes. Su comprobación integral no forma parte de las 55 aserciones nuevas.

Permisos: operaciones especiales y «todo pagado» requieren ADMIN. La API histórica de marcado individual y algunas consultas admiten TEACHER mediante `requireTeacher`. No se ha cambiado esa política ni debe deducirse un permiso de la apariencia de la interfaz.

ENTREGA TÉCNICA · DATOS

17 · ANTES DE USAR EL BACKEND ACTUALIZADO

Migración aditiva pendiente

El código necesita dos columnas nuevas. Sin ellas, el cliente actualizado puede fallar al consultar los modelos correspondientes. Añadirlas al esquema de Prisma no las crea por sí solo en una base ya existente. Se validó el cambio en PostgreSQL 15 aislado; aplicarlo al entorno real requiere respaldo, destino confirmado y ventana de despliegue.

```
ALTER TABLE "TermGrade"
ADD COLUMN IF NOT EXISTS "gradeOverrides"
JSONB NOT NULL DEFAULT '{}';

ALTER TABLE "TaskDelivery"
ADD COLUMN IF NOT EXISTS "gradeMode" TEXT;
```

`gradeOverrides`	Objeto JSON de ajustes por campo. Por defecto vacío: no fuerza todas las notas antiguas a modo manual.
`gradeMode`	Texto AUTO/MANUAL validado en la ruta. El nulo identifica datos históricos tratados de forma compatible.
SQL repetible	`IF NOT EXISTS` evita volver a crear una columna existente. No borra tablas ni realiza un backfill masivo.

Secuencia que se ejecutó en aislamiento

1. Crear PostgreSQL temporal separado, sin volumen ni conexión a la base de HitSchool.
2. Sincronizar el esquema actual en la base vacía, sin `--accept-data-loss`.
3. Ejecutar dos veces el SQL aditivo en el esquema nuevo.
4. Probar también las columnas sobre tablas previas con filas históricas; verificar valores conservados y defaults.
5. Ejecutar las rutas con Prisma real y notificaciones SMTP sustituidas por funciones locales.

```
Desde backend/, con el destino verificado:
npx prisma db execute --file prisma/add-grade-overrides.sql \
  --schema prisma/schema.prisma
npx prisma generate
```

No usar el arranque de desarrollo como migración segura

El script `backend dev` ejecuta `schema:sync`, que contiene `prisma db push --accept-data-loss`. No debe utilizarse para actualizar a ciegas una base con datos. En el PostgreSQL Docker local, las columnas ya existían, por lo que no se ejecutó el SQL aditivo; sí se añadieron solo fixtures rotulados. No se ejecutaron semillas generales ni resets.

VERIFICACIÓN · EVIDENCIA DISPONIBLE

19 · QUÉ SE HA COMPROBADO

Pruebas simuladas y reales

Prueba	Evidencia y límite
Notas HTTP simuladas	ADMIN/TEACHER × presencial/online; PUT/GET, competencias, cero, ajustes, reset, feedback y permisos. Router y JWT reales, Prisma simulado.
Chat HTTP simulado	Alumno/tutor frente a profesor/admin en ambos sentidos, hijos y llegada concurrente.
Pagos: 55 aserciones	Servicio y rutas con datos simulados: tarifas, septiembre, verano, cobros, cuotas ajenas/exentas y reversión del alta.
PostgreSQL 15 aislado	Persistencia de TermGrade, TaskDelivery, ChatMessage, PaymentStatus, altas mensuales/trimestrales. Cinco grupos de integración pasan; SMTP sustituido.
SQL sobre legado	Columnas nuevas repetibles y filas históricas conservadas en una segunda base desechable de esquema previo.
Navegador: 16 escenarios	Chromium real; 1366×900 y 390×844; notas/tareas y chat. API interceptada.
Dos boletines PDF	Se genera presencial y online, comprobando etiqueta y valores manuales, incluido cero.
Validación estática	TypeScript backend, build frontend, validación del esquema y diff de espacios: pasan.

Comandos reproducibles desde la raíz

```
npx --prefix backend ts-node --project backend/tsconfig.json backend/test-academic-flows.ts
npx --prefix backend ts-node --project backend/tsconfig.json backend/test-chat-flows.ts
npx --prefix backend ts-node --project backend/tsconfig.json backend/test-payment-rules.ts
npx --prefix backend tsc --project backend/tsconfig.json --noEmit
npm --prefix frontend run build
```

Para navegador, instalar una vez `npx --prefix frontend playwright install chromium`; iniciar el frontend en una terminal y las pruebas en otra:

```
npm --prefix frontend run dev -- --host 127.0.0.1 --port 4173 --strictPort
npm --prefix frontend run test:flows
```

Capturas y PDF de prueba: directorio temporal del sistema `hitschool-flow-checks`. El log `Simulated payment failure` es intencionado: prueba la reversión del alta. El `npm test` original del backend es un placeholder; no es la suite usada aquí.

La frontera de esta evidencia

Las pruebas PostgreSQL de integración usaron una base temporal sin volumen y SMTP simulado. Además se cargaron fixtures rotulados en la base Docker local para la comprobación manual web. Esto no demuestra el estado del servidor remoto ni la migración en producción. No se han probado cargas concurrentes, reinicio durante una operación ni todos los datos históricos.

20 · ESCENARIO YA PREPARADO

Qué buscar en la web

Los accesos precargados no se tocaron

El seed no actualiza usuarios, roles, hashes ni contraseñas. Usa las credenciales ya publicadas en la guía de despliegue. El correo `marpargut@hitschool.com` corresponde a dos usuarios activos diferentes, uno `STUDENT` y otro `PARENT`.

Pantalla	Escenario y resultado inicial esperado
Mis clases → Calificaciones	`PRUEBA CAMBIOS 2026-11 · Clase presencial`; T1, 2025-2026. Midterm 8, Final 9, tareas manual 7,25, global automática 8,13.
Expediente → Historial	Tarea MANUAL 7,5 con paso 8,5; tarea AUTO 6. Al reiniciar la primera, toma la nota del paso.
Mensajes	Los cuatro roles tienen al menos un mensaje entrante demo sin leer. El tutor recibe mensajes asociados a su hijo.
Gestión de matrículas	`2026-2027`: octubre–diciembre, mensual 65 €. `2027-2028`: octubre–diciembre, trimestral 60 €/mes.
Pagos	Seis cuotas demo pendientes: tres a 65 € y tres a 60 €. Marcar y desmarcar una para comprobar ambos estados.

Comprobaciones sugeridas

1. Entrar como `profesor@hitschool.com` y cambiar una nota, guardarla y volver a abrirla. Probar también el retorno a automática.
2. Entrar como admin para comprobar permisos de edición y el selector de periodicidad.
3. Entrar como alumno y como tutor con el rol correspondiente para comprobar lectura y chat pendientes.
4. En matrículas, seleccionar cada curso escolar y verificar los importes mensuales y trimestrales.
5. En pagos, marcar y desmarcar; la cuota y su fecha deben quedar registradas en el entorno local.

Carga y limpieza local

```
Desde la raíz, PowerShell:
$databaseUrl = docker exec hit_school_backend printenv DATABASE_URL
$databaseUrl = $databaseUrl -replace '@db:5432/', '@127.0.0.1:5432/'
$env:DATABASE_URL = $databaseUrl
npx --prefix backend ts-node --project backend/tsconfig.json backend/seed-change-verification-data.ts

Para quitar solo los registros marcados, sin borrar cuentas:
npx --prefix backend ts-node --project backend/tsconfig.json backend/cleanup-change-verification-
data.ts
```

Los scripts solo admiten `127.0.0.1:5432/hitschool`. No los apuntes al servidor. El seed es idempotente y etiqueta sus filas; cleanup borra solo la clase, sus pagos y mensajes rotulados. El backend Docker local arranca con `db push --accept-data-loss`: evita reiniciarlo como forma de migrar bases con datos.

21 · SIN CONFIRMACIONES POR SUPOSICIÓN

Qué no debe darse por hecho

Bloqueos y exclusiones conocidos

- **Servidor:** la persistencia se probó en PostgreSQL temporal y la interfaz usa fixtures en Docker local. No se ha tocado la base del servidor remoto.
- **Mitad de trimestre:** no se ha modificado; el salto posterior de calendario puede omitir meses. No se presenta como resuelto.
- **Históricos y reactivaciones:** libros de cursos anteriores, exenciones y posibles carreras de generación necesitan una revisión específica con datos representativos.
- **Mensajes antiguos sin alumno:** no se han reclasificado masivamente ni se garantiza que todos los historiales legados encajen en los nuevos contextos.
- **Permisos financieros:** las rutas que históricamente admiten TEACHER no se han convertido silenciosamente en ADMIN-only.

Advertencias ajenas a las correcciones funcionales

Comprobación	Resultado que debe conservarse en el informe
Lint global	Tres errores de `rules-of-hooks` en `MaterialsManagement.tsx`, pantalla no modificada; también existen advertencias en varios archivos. No está limpio.
Build	Pasa, con aviso de tamaño de bundle superior a 500 kB. No se ha introducido una reorganización del empaquetado.
Auditoría npm	Avisos en `dompurify` (crítico), `source-map-js`, `vite` y `xlsx` (altos). No se han aplicado actualizaciones automáticas fuera del alcance.

Antes de aplicar al servidor

1. Preparar y verificar una copia de seguridad.
2. Confirmar explícitamente `DATABASE_URL` y el destino.
3. Aplicar solo el SQL aditivo en ventana de despliegue controlada; no usar `schema:sync` con `--accept-data-loss`.
4. Verificar columnas, prueba de lectura y escritura acotada, logs y restauración prevista.
5. Informar por separado del resultado en el destino real.

Se ha mantenido la separación entre «cambio implementado», «prueba PostgreSQL desechable pasada» y «funcionamiento verificado en el servidor real». Solo las dos primeras están acreditadas.

22 · DÓNDE ESTÁ CADA RESPONSABILIDAD

Archivos y puntos de entrada

Archivo o grupo	Responsabilidad en estos cambios
`backend/src/routes/termGrades.ts`	Resolución común, ajustes, competencias y medias; `createTermGradesRouter`.
`backend/src/routes/assignments.ts`	Nota individual, feedback sin borrado y acceso por clase; factoría de pruebas.
`backend/src/routes/structuredTasks.ts`	Sincronización global; `createTaskDeliveryGradingHandler`, pasos y modo de tarea.
`backend/src/routes/chat.ts`	Contexto tutor/admin, lectura por IDs y contador restante; `createChatRouter`.
`backend/src/routes/enrollments.ts`	Alta transaccional y SMTP posterior; `createEnrollmentsRouter`.
`backend/src/routes/students.ts`	La edición de tarifa deja de sobrescribir cargos especiales.
`backend/src/routes/payments.ts`	Identidad cuota/alumno y protección de exentos; `createPaymentsRouter`.
`backend/src/services/payments.ts`	Cliente transaccional y exclusión de nuevas cuotas ordinarias de julio/agosto.
`backend/prisma/schema.prisma` `backend/prisma/add-grade-overrides.sql`	Campos nuevos; SQL probado en PostgreSQL aislado. El esquema Docker local ya incluye columnas; servidor remoto pendiente.
`frontend/src/components/ExpedienteAcademico.tsx`	Editor, ajustes, reset, errores, notas de exámenes y apertura de evaluación global de tareas.
`frontend/src/components/TaskDeliveryReviewModal.tsx`	Pasos, modo global, reset y controles adaptados a móvil.
`frontend/src/components/GradesTab.tsx`	Etiqueta y contrato de guardado de pasos/modo conservados en esta superficie.
`frontend/src/components/StudentGradesTab.tsx` `frontend/src/pages/StudentGrades.tsx`	Etiqueta Midterm en lectura de alumno/familia.
`frontend/src/pages/Chat.tsx`	Negrita, carga visible, IDs, protección frente a respuestas tardías y contexto de hijo.
`frontend/src/utils/reportCard.ts`	Midterm y texto neutral de evaluación continua en el boletín.

`backend/test-academic-flows.ts`
`backend/test-chat-flows.ts`
`backend/test-payment-rules.ts`
`backend/test-postgres-integration.ts`
	Pruebas HTTP/servicios simuladas y una integración adicional con PostgreSQL efímero real.

`backend/seed-change-verification-data.ts` `backend/cleanup-change-verification-data.ts`	Seed idempotente y limpieza segura de fixtures de la base Docker local; no toca accesos.
`frontend/test-flows.mjs` `frontend/package.json` y lockfile	Playwright, comando reproducible, capturas y dos PDF.
`DEPLOYMENT_WALKTHROUGH.md`	Migración, reproducción de pruebas, advertencias y límites.

Terminología: automática = cálculo vigente; manual = ajuste explícito; efectiva = valor que prevalece; pendiente = obligación no cobrada; exenta = obligación dispensada; transacción = grupo de escrituras que confirma o revierte conjuntamente.